

Le superfici con OpenGL

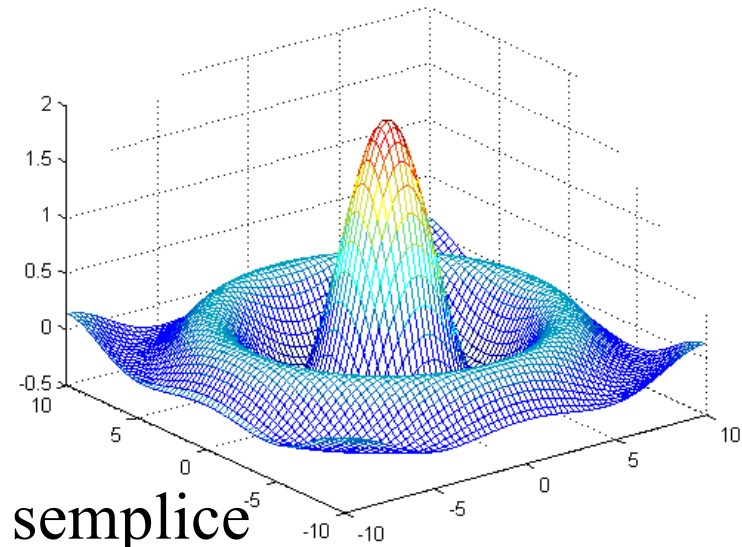
Computer Graphics 2025/2026

Rappresentazione non parametrica esplicita

$$z = f(x, y) \quad (x, y) \in [a, b] \times [c, d]$$

Esempio

$$z = \frac{2\sin(\sqrt{x^2 + y^2})}{\sqrt{x^2 + y^2}} \quad (x, y) \in [-10, 10] \times [-10, 10]$$



- Il calcolo di punti sulla superficie è semplice
- Fornisce un unico valore della variabile dipendente per ciascuna coppia di valori delle variabili indipendenti
 - Non consente di rappresentare superfici chiuse o a più valori (es. sfera, ...)

Rappresentazione non parametrica implicita

$$f(x, y, z) = 0 \quad (x, y, z) \in [a, b] \times [c, d] \times [e, f]$$

Esempio

Sfera di raggio r con centro nell'origine

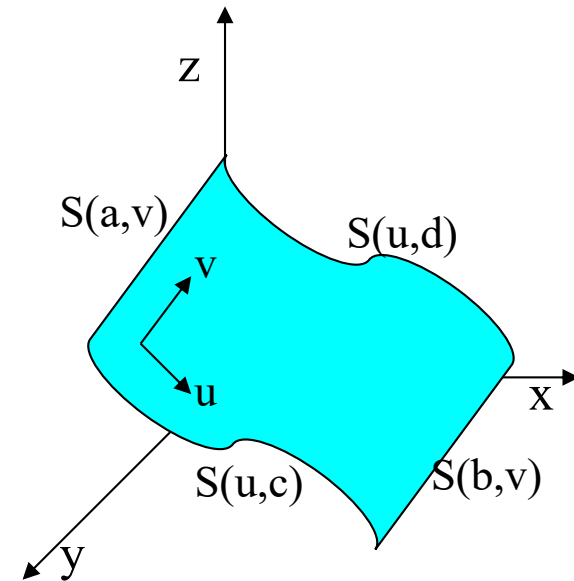
$$x^2 + y^2 + z^2 - r^2 = 0 \quad (x, y, z) \in \mathbb{R}^3$$

- Consente di rappresentare tutte le superfici
- Il calcolo di punti sulla superficie è in genere difficile

Rappresentazione parametrica

$$\begin{cases} x = x(u, v) \\ y = y(u, v) \\ z = z(u, v) \end{cases} \quad (u, v) \in [a, b] \times [c, d]$$

$$S(u, v) = [x(u, v), y(u, v), z(u, v)]^T$$

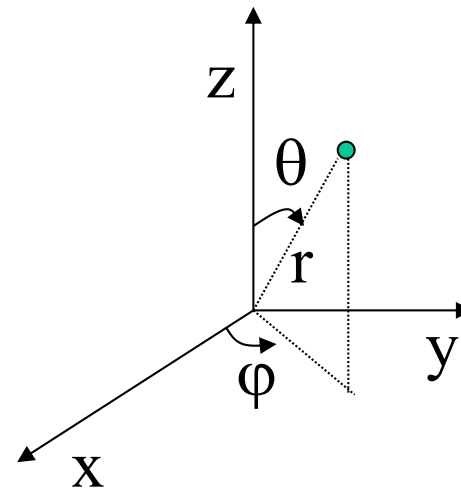


Rappresentazione parametrica: esempio

Sfera di raggio r con centro nell'origine

$$\begin{cases} x = x(\theta, \varphi) = r \sin\theta \cos\varphi \\ y = y(\theta, \varphi) = r \sin\theta \sin\varphi \\ z = z(\theta, \varphi) = r \cos\theta \end{cases} \quad (\theta, \varphi) \in [0, \pi] \times [-\pi, \pi]$$

$(\theta, \varphi) = \textit{coordinate sferiche}$



Rappresentazione parametrica

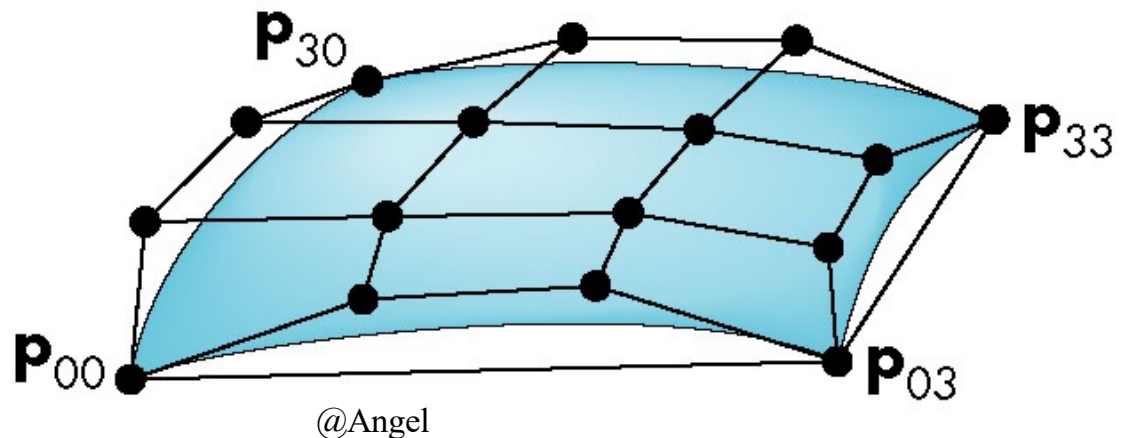
- È esplicita → Il calcolo di punti della superficie è semplice
- Consente di rappresentare superfici chiuse o con valori multipli
- Adatta alla formulazione di superfici composite (unione di più superfici)
- Consente di esprimere una superficie come combinazione lineare di funzioni scalari dei parametri, con coefficienti vettoriali:

$$S(u, v) = \sum_{i=0}^n \sum_{j=0}^m c_{i,j} f_{i,j}(u, v) \quad (u, v) \in [a, b] \times [c, d]$$

Superfici di Bezier

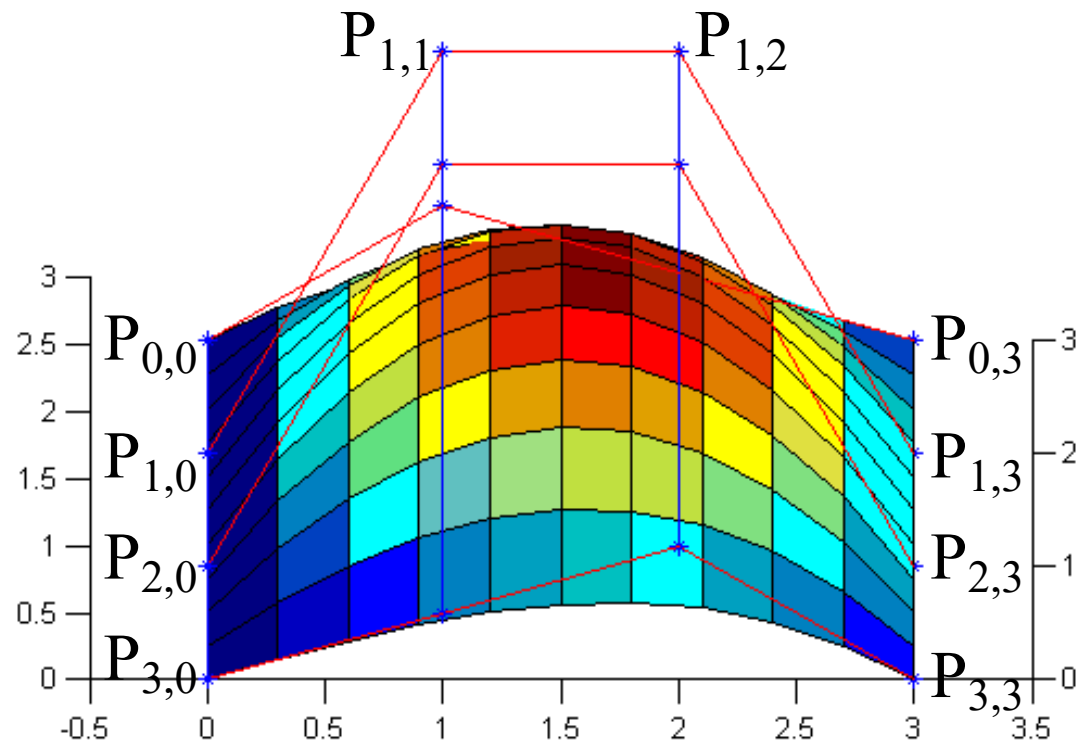
$$S(u, v) = \sum_{i=0}^n \sum_{j=0}^m P_{i,j} B_{i,n}(u) B_{j,m}(v) \quad (u, v) \in [0,1] \times [0,1]$$

- $P_{i,j}$, $i=0, \dots, n$, $j=0, \dots, m$ *punti di controllo* (formano il *poliedro di controllo*)
- $B_{i,n}(u)$, $B_{j,m}(v)$ polinomi di Bernstein di grado n e m
- (n,m) grado della superficie



Esempio: superficie di Bezier di grado (3,3)

$$S(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 P_{i,j} B_{i,3}(u) B_{j,3}(v) \quad (u, v) \in [0,1] \times [0,1]$$



Proprietà delle superfici di Bezier

- *Interpolazione*: $S(u,v)$ interpola i punti d'angolo del poliedro di controllo:

$$S(0,0)=P_{0,0}, S(1,0)=P_{n,0}, S(0,1)=P_{0,m}, S(1,1)=P_{n,m}$$

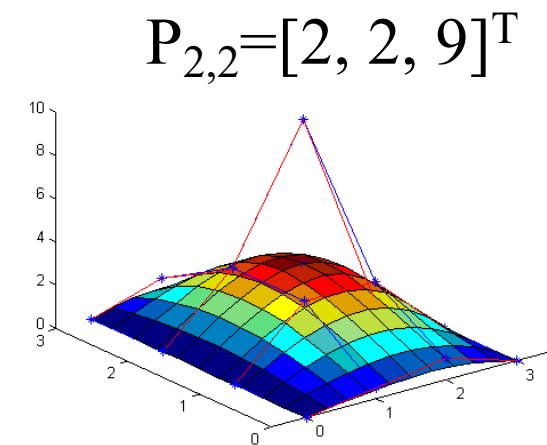
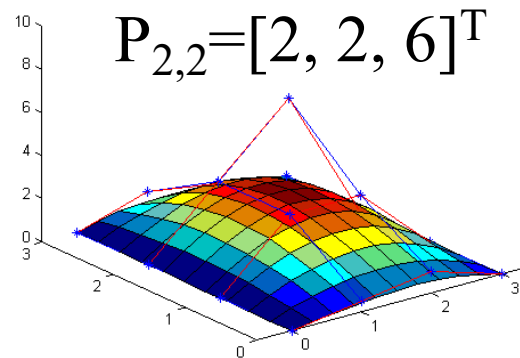
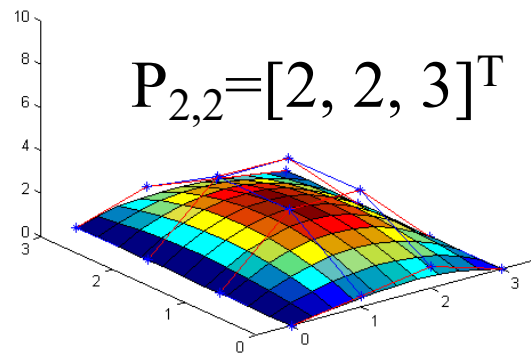
- *Convex hull*: giace completamente all'interno del più piccolo insieme convesso contenente i punti di controllo
- *Invarianza affine*: la relazione fra la superficie e i punti di controllo è invariante per trasformazioni affini
- *Precisione lineare*: se tutti i punti di controllo sono su un piano, la superficie giace sul piano stesso
- Le 4 curve di bordo sono curve di Bezier:

$$S(u,0) = \sum_{i=0}^n P_{i,0} B_{i,n}(u)$$

e analogamente per $S(u,1)$, $S(0,v)$, $S(1,v)$

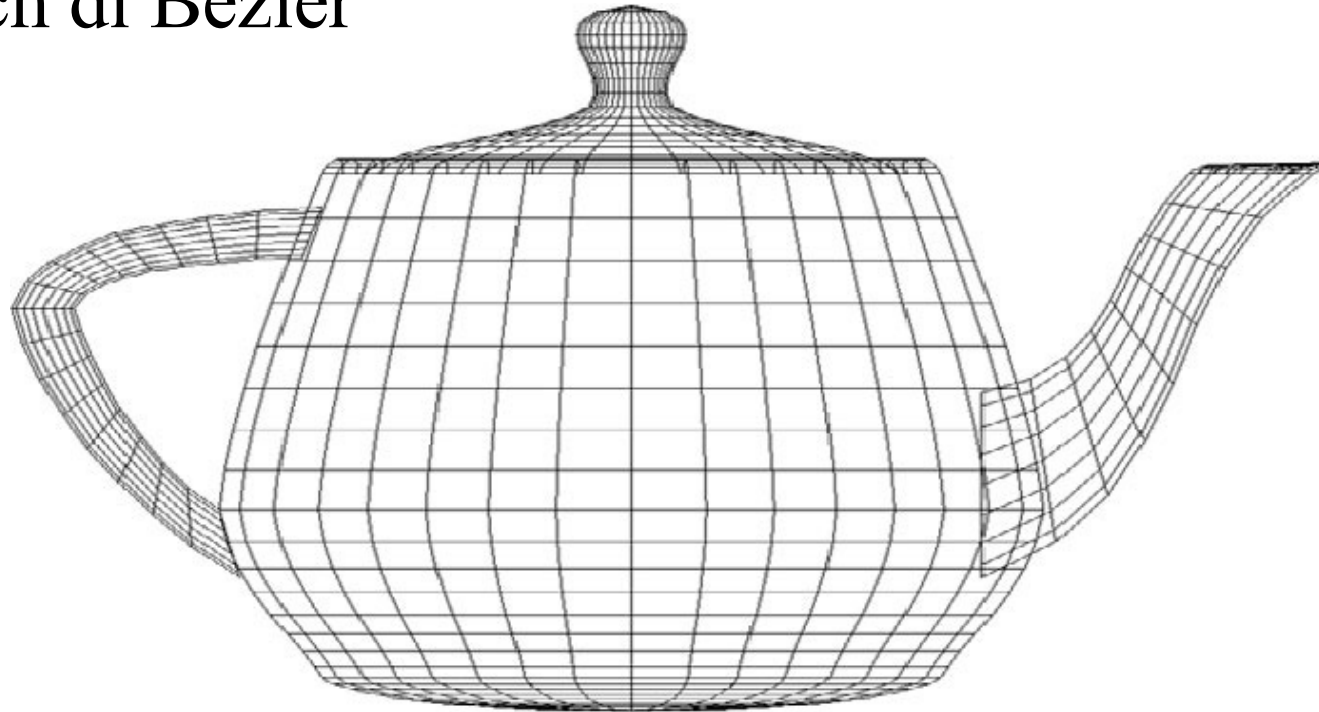
Proprietà delle superfici di Bezier (cont.)

- *Controllo globale*: spostando un punto di controllo la forma della superficie viene interamente modificata



Utah teapot

- Uno dei più famosi insiemi di dati in Computer Graphics
- Lista di 306 vertici 3D e indici che definiscono 32 patch di Bezier



Superfici di Bezier in OpenGL

“Valutatori” 2D, definiti mediante:

```
glMap2f(type, u_min, u_max, u_stride, u_order,  
          v_min, v_max, v_stride, v_order, pts);
```

- type = GL_MAP2_VERTEX_3 (tipo di oggetto da valutare)
- u_min, u_max (v_min, v_max)= range del parametro u (v)
- u_stride (v_stride)= numero di valori f.p. fra un punto di controllo ed il successivo nella direzione u (v)
- u_order (v_order) = ordine della curva nella direzione u (v)
- &pts[0][0] = puntatore al primo punto di controllo

Superfici di Bezier in OpenGL (cont.)

Un valutatore deve essere prima abilitato:

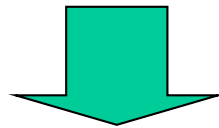
glEnable(type);

- `type` = tipo di oggetto da valutare.

Una volta definito ed abilitato un valutatore, per utilizzare un valore dal valutatore attivo si può usare:

glEvalCoord2f(u,v);

- `u (v)`= valore del parametro `u (v)` in cui valutare l'oggetto



I vertici della superficie possono essere usati come ogni altro vertice

Superfici di Bezier in OpenGL: esempio

```
GLfloat cps[4][4][3] = {{{-1.5,-1.5,4.0},{-0.5,-1.5,2.0},{0.5,-1.5,-1.0},{1.5,-1.5,2.0}},  
                        {{-1.5,-0.5,1.0},{-0.5,-0.5,3.0},{0.5,-0.5,0.0},{1.5,-0.5,-1.0}},  
                        {{-1.5,0.5,4.0},{-0.5,0.5,0.0},{0.5,0.5,3.0},{1.5,0.5,4.0}},  
                        {{-1.5,1.5,-2.0},{-0.5,1.5,-2.0},{0.5,1.5,0.0},{1.5,1.5,-1.0}}};
```

...

```
glMap2f(GL_MAP2_VERTEX_3, 0, 1, 3, 4, 0, 1, 12, 4, &cps[0][0][0]);  
glEnable(GL_MAP2_VERTEX_3);
```

...

```
for (j = 0; j <= 8; j++) {  
    glBegin(GL_LINE_STRIP);  
        for (i = 0; i <= 30; i++) glEvalCoord2f((GLfloat)i/30.0, (GLfloat)j/8.0);  
    glEnd();  
    glBegin(GL_LINE_STRIP);  
        for (i = 0; i <= 30; i++) glEvalCoord2f((GLfloat)j/8.0, (GLfloat)i/30.0);  
    glEnd();  
}
```

Superfici di Bezier in OpenGL (cont.)

Se il valutatore attivo deve essere utilizzato per valutare l'oggetto per un insieme di $n*m$ valori equispaziati dei parametri, si può usare:

```
glMapGrid2f(n, u1, u2, m, v1, v2);
```

```
glEvalMesh2(GL_POINT, i1, i2, j1, j2);
```

```
/* o GL_LINE o GL_FILL */
```

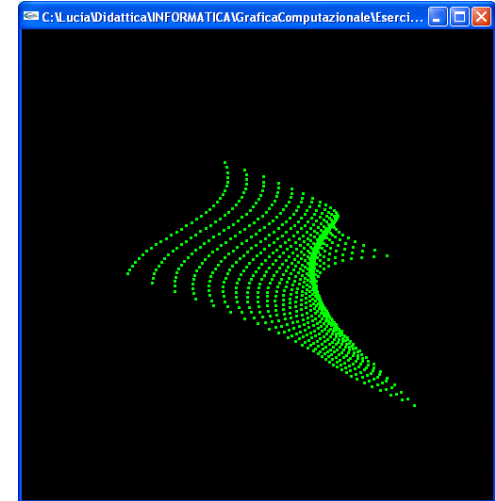
- $0 \leq i1, i2 \leq n$
- $0 \leq j1, j2 \leq m$

Superfici di Bezier in OpenGL (cont.)

```
glMapGrid2f(n, u1, u2, m, v1, v2);  
glEvalMesh2(GL_POINT, 0, n, 0, m);
```

equivale a:

```
glBegin(GL_POINTS);  
  for (i = 0; i <= n; i++)  
    for (j = 0; j <= m; j++)  
      glEvalCoord2f(u1+i*(u2-u1)/n,  
                   v1+j*(v2-v1)/m);  
glEnd();
```

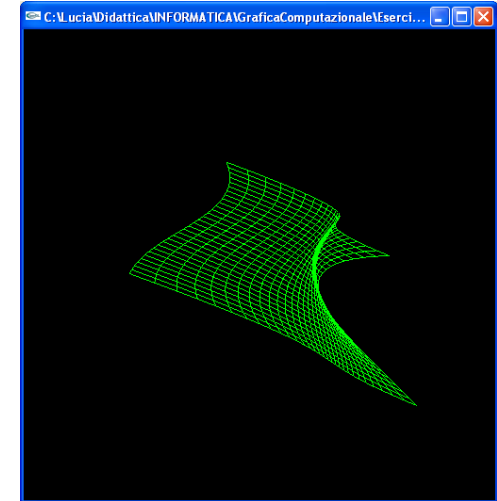


Superfici di Bezier in OpenGL (cont.)

```
glMapGrid2f(n, u1, u2, m, v1, v2);  
glEvalMesh2(GL_LINE, 0, n, 0, m);
```

equivale a:

```
for (i = 0; i <= n; i++){  
    glBegin(GL_LINE_STRIP);  
        for (j = 0; j <= m; j++) glEvalCoord2f(u1+i*(u2-u1)/n,  
                                                  v1+j*(v2-v1)/m);  
    glEnd();}  
for (j = 0; j <= m; j++) {  
    glBegin(GL_LINE_STRIP);  
        for (i = 0; i <= n; i++) glEvalCoord2f(u1+i*(u2-u1)/n,  
                                                  v1+j*(v2-v1)/m);  
    glEnd();}
```

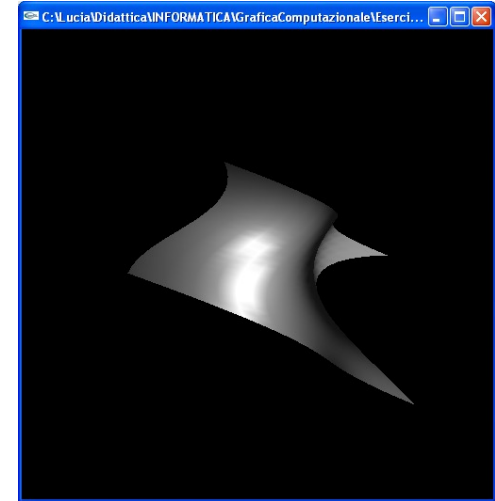


Superfici di Bezier in OpenGL (cont.)

```
glMapGrid2f(n, u1, u2, m, v1, v2);  
glEvalMesh2(GL_FILL, 0, n, 0, m);
```

equivale a:

```
for (i = 0; i < n; i++){  
    glBegin(GL_QUAD_STRIP);  
        for (j = 0; j <= m; j++) {  
            glEvalCoord2f(u1+i*(u2-u1)/n, v1+j*(v2-v1)/m);  
            glEvalCoord2f(u1+(i+1)*(u2-u1)/n, v1+j*(v2-v1)/m); }  
    glEnd();}
```



Esercizio 1

- Scrivere un programma (con OpenGL) per il rendering di una superficie di Bezier di ordine (4,4) utilizzando le funzioni di OpenGL

```
GLfloat ctrlpoints[4][4][3] = {  
    {{0, 0, 0},{1, 0, 0.5},{2, 0, 1},{3, 0, 0}},  
    {{0, 1, 0},{1, 1, 3},{2, 1, 3},{3, 1, 0}},  
    {{0, 2, 0},{1, 2, 3},{2, 2, 3},{3, 2, 0}},  
    {{0, 3, 0},{1, 3, 1},{2, 3, 0.5},{3, 3, 0}}};
```

Suggerimento per GL_FILL

- Abilitazione del calcolo automatico delle normali:

```
glEnable(GL_AUTO_NORMAL);
```

Superfici di Bezier razionali

$$S(u, v) = \frac{\sum_{i=0}^n \sum_{j=0}^m w_{i,j} P_{i,j} B_{i,n}(u) B_{j,m}(v)}{\sum_{i=0}^n \sum_{j=0}^m w_{i,j} B_{i,n}(u) B_{j,m}(v)} \quad (u, v) \in [0,1] \times [0,1]$$

Strategia generale per algoritmi relativi alle superfici razionali

1. Calcolo delle coordinate omogenee $P_{i,j}^W$ dei punti di controllo $P_{i,j}$
2. Applicazione dell'algoritmo alla superficie non razionale con punti di controllo $P_{i,j}^W$
3. Proiezione (omogeneizzazione) del risultato

Superfici di Bezier razionali in OpenGL: esempio

```
GLfloat cp[4][4][3] = {{{-1.5,-1.5,4.0},{-0.5,-1.5,2.0},{0.5,-1.5,-1.0},{1.5,-1.5,2.0}},  
                        {{-1.5,-0.5,1.0},{-0.5,-0.5,3.0},{0.5,-0.5,0.0},{1.5,-0.5,-1.0}},  
                        {{-1.5,0.5,4.0},{-0.5,0.5,0.0},{0.5,0.5,3.0},{1.5,0.5,4.0}},  
                        {{-1.5,1.5,-2.0},{-0.5,1.5,-2.0},{0.5,1.5,0.0},{1.5,1.5,-1.0}}};
```

```
GLfloat w[4][4]={{1, .5, .5, 1},{1, .5, .5, 1},{1, .5, .5, 1},{1, .5, .5, 1}};
```

```
GLfloat cw[4][4][4];
```

```
...
```

“Calcolo cw (coordinate omogenee dei cp)”

```
...
```

```
glMap2f(GL_MAP2_VERTEX_4, 0, 1, 4, 4, 0, 1, 16, 4, &cw[0][0][0]);
```

```
glEnable(GL_MAP2_VERTEX_4);
```

```
...
```

```
glMapGrid2f(30, 0.0, 1.0, 30, 0.0, 1.0);/*inizializza i valori equispaziati*/
```

```
glEvalMesh2(GL_POINT, 0, 30, 0, 30); /* o GL_LINE o GL_FILL */
```

Esercizio 2

- Scrivere un programma (con OpenGL) per il rendering di una superficie di Bezier razionale di ordine (4,4) utilizzando le funzioni di OpenGL

```
GLfloat ctrlpoints[4][4][3] = {  
    {{0, 0, 0},{1, 0, .5},{2, 0, 1},{3, 0, 0}},  
    {{0, 1, 0},{1, 1, 3},{2, 1, 3},{3, 1, 0}},  
    {{0, 2, 0},{1, 2, 3},{2, 2, 3},{3, 2, 0}},  
    {{0, 3, 0},{1, 3, 1},{2, 3, 0.5},{3, 3, 0}}};
```

```
GLfloat pesi[4][4]={{{1, .5, .5, 1},{1, .5, .5, 1},  
                    {1, .5, .5, 1},{1, .5, .5, 1}}};
```